Laboratório 1

Objetivos:

- Criar conta e projeto na Google Cloud Platform (GCP)
- Criar máquinas virtuais (VM) na Google Cloud Platform
- Aceder remotamente a uma VM através de um cliente Secure Socket Shell (SSH)
- Execução de um artefacto Java numa VM acessível via HTTP em diferentes portos TCP/IP usando *Firewall rules*.

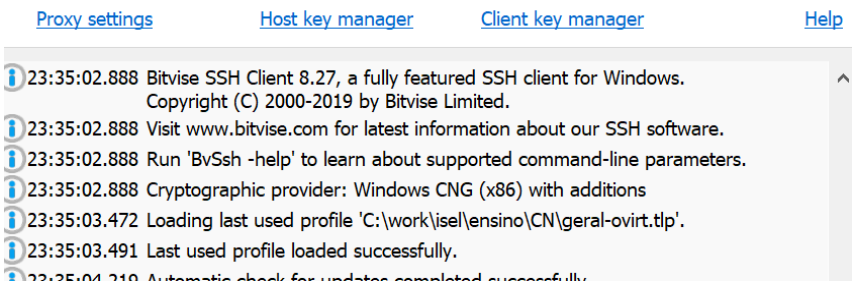
1) Seguindo o guião "***CN-Registo na Google Cloud Platform como Aluno - Verao2526.pdf***" garanta que acede à sua conta e projeto na consola web do GCP.

   O ponto 2 assume computadores pessoais com sistema operativo Windows. Para computadores com Linux ou MacOS realizar a ações indicadas em Anexo no fim do documento.

2) As máquinas virtuais que vão ser criadas no GCP são acedidas via SSH com autenticação de chave pública e privada. A aplicação cliente SSH que se recomenda para o Windows é o Bitvise (https://www.bitvise.com/ssh-client-download).

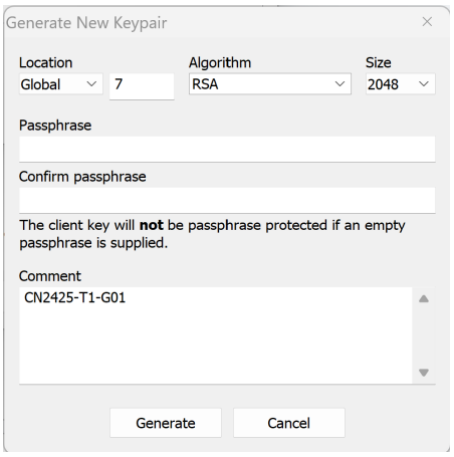
   As alíneas seguintes mostram como cada aluno pode gerar um par de chaves pública/privada com o cliente SSH Bitvise em Windows:

   a) No cliente Bitvise aceda a "Client Key Manager":

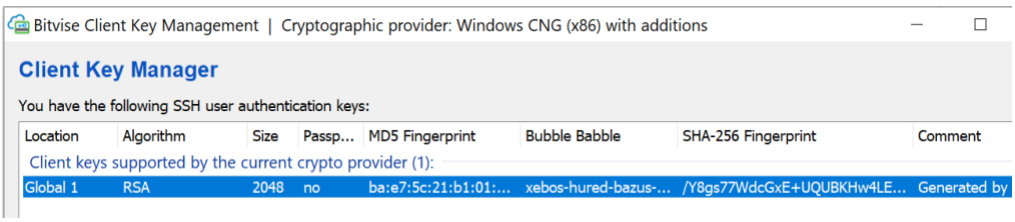


   b) Na zona inferior da janela, escolha "*Generate New*"

   c) Escolha uma password para proteger a chave privada, ou deixe em branco. Na caixa de comentário ("*Comment*") indique um identificador com o formato CN2526-<turma>-<grupo>. Use o nome do grupo e turma como no projeto GCP, por exemplo, CN2425-T1-G01. Note que cada aluno deverá ter um par de chaves diferentes, mas deve usar o mesmo identificador, o qual será o nome de utilizador a usar na sessão SSH para a VM, permitindo assim que todos os alunos partilharem a mesma *home directory* no Linux da VM.



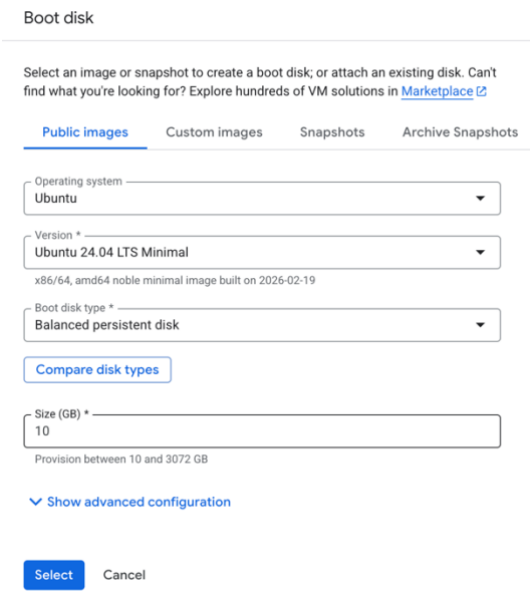
d) Selecione "*Generate*" para gerar o par de chaves e acrescentar à lista de chaves disponíveis no cliente Bitvise:



e) Exporte a chave pública escolhendo a opção "*Export"* da mesma janela. Indique o formato "*OpenSSH*" e exporte a chave pública para um ficheiro e diretoria à sua escolha no seu computador pessoal.

f) Visualize a chave pública exportada com um editor de texto (ex: VS Code, Notepad, …). O formato da chave deve ter três partes: `ssh-rsa <chave> <identificador do grupo>`. Caso falte a última parte, complete com o identificador que introduziu no campo *Comment* na alínea c).

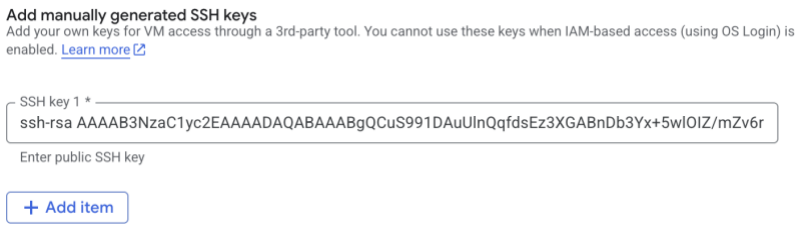
3) Usando a conta GCP do grupo de alunos, no serviço *Compute Engine* crie 1 instância de máquina virtual: atribua um nome e região, selecione o tipo (*Series E2 Machine Type 'e2-small'*) e o sistema operativo (*Boot Disk*) Ubuntu 24.04 LTS (arquitetura x86/64).



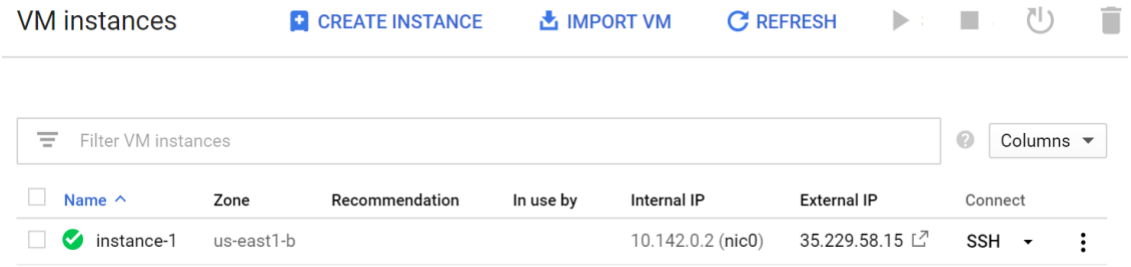
a. Para permitir ligações ao porto 80 da VM, ative, no menu "*Networking*", a opção HTTP na *Firewall*.

b. Em "*Security*", extenda a opção "*Manage Access*" e adicione um item "+ *Add item*" na opção de "*Add manually generated SSH keys"*.

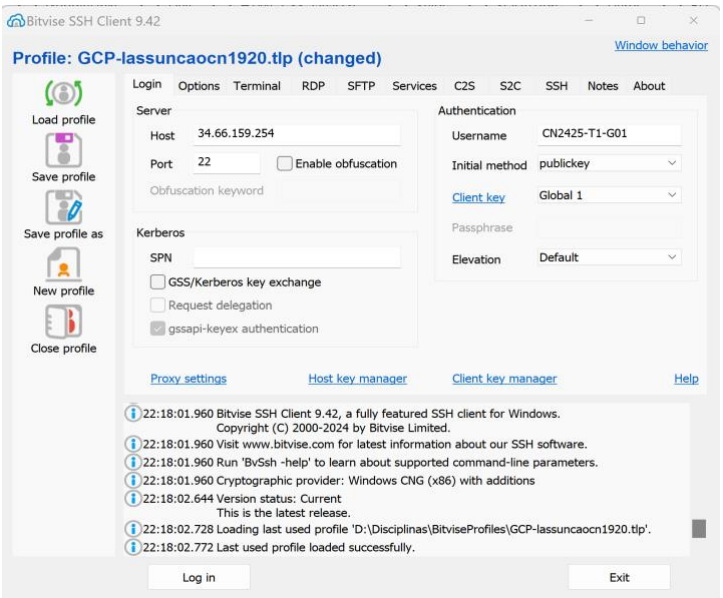
   Copie integralmente para a caixa de texto disponível a chave pública SSH gerada e exportada anteriormente, garantindo que no final fica o identificador que será o nome de utilizador a usar na sessão SSH para a VM. Embora possa posteriormente adicionar as chaves dos restantes alunos do grupo, pode já nesta fase adicionar as várias chaves.



c. Crie a VM e verifique na consola Web do GCP que a máquina foi iniciada e tem um IP externo (endereço IP público) acessível em qualquer localização:



d. Aceda à VM através do cliente SSH (ver figura seguinte). O utilizador é o indicado anteriormente (ex: CN2425-T1-G01) o método inicial é "*public key*" e a "*Client key*" tem de indicar a entrada correta (ex: *Global 1*).



e. Após *login*, verifique o correto acesso à VM. Não se esqueça de desligar a VM quando não a estiver a usar, usando o botão "*Stop*" na consola Web do GCP. Para ver a chave instalada na VM pode executar o seguinte comando Linux: `cat .ssh/authorized_keys`.

4) Com permissões de *super user*, faça *update* aos *packages* do Ubuntu: `sudo apt update`

5) Instale o *runtime* Java JDK 25, usando o comando com permissões de *super user:*
   `sudo apt install openjdk-25-jdk`

6) Em anexo ao enunciado existe o artefacto java `ServiceREST.jar`, que implementa um serviço HTTP REST, por omissão no porto 80, mas que ao ser lançado suporta a passagem como argumento outro porto TCP/IP (`java -jar ServiceREST.jar [porto]`). O serviço disponibiliza as seguintes rotas:

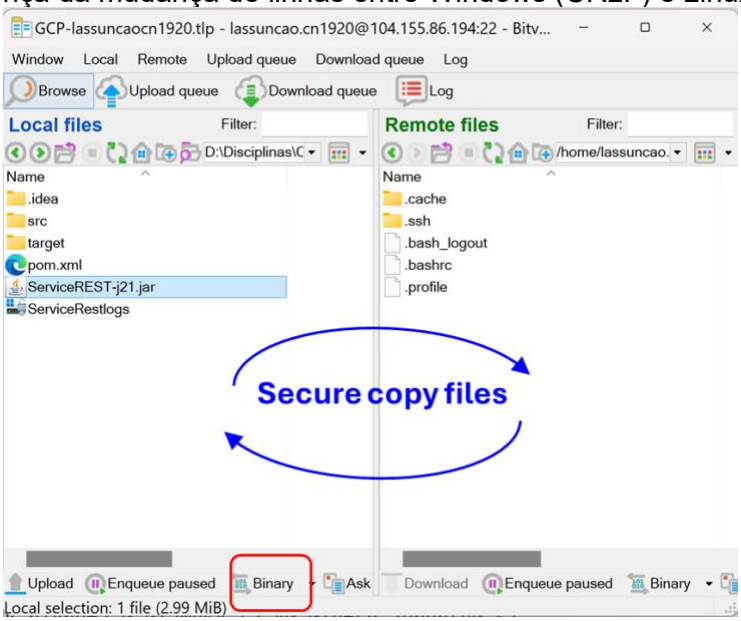
```
http://<host ip>[:porto]/ping
http://<host ip>[:porto]/hello/<some name>
http://<host ip>[:porto]/calc/number{+,-,*}number
      Ex: http://<host ip>[:porto]/calc/5*3 retorna 15
```

7) Execute o serviço localmente na sua máquina com diferentes portos e usando um browser HTTP aceda ao serviço experimentando as diferentes rotas, indicando como <*host ip*> o endereço *localhost*.

8) Verifique que o serviço escreve um ficheiro de texto de nome `ServiceRestlogs`, na diretoria onde se executa o serviço, com logs sobre os pedidos realizados ao serviço.

9) Faça *upload* do JAR (`ServiceREST.jar`) para a VM na GCP. Execute na VM o serviço com o porto por omissão 80, com o seguinte comando: `sudo java -jar ServiceREST.jar`. (Note que precisa de executar como *super user* por restrições do sistema operativo Linux na utilização do porto 80).

   Note que o cliente Bitvise tem a opção New *SFTP window* para fazer "*Secure Copy*" entre a máquina local e a VM no GCP ou entre a VM no GCP e a sua máquina local. Deve usar-se opção *Binary* quando se copiam ficheiros executáveis e quando forem ficheiros de texto deve usar-se a opção Text LF devido à diferença da mudança de linhas entre Windows (CRLF) e Linux (LF).

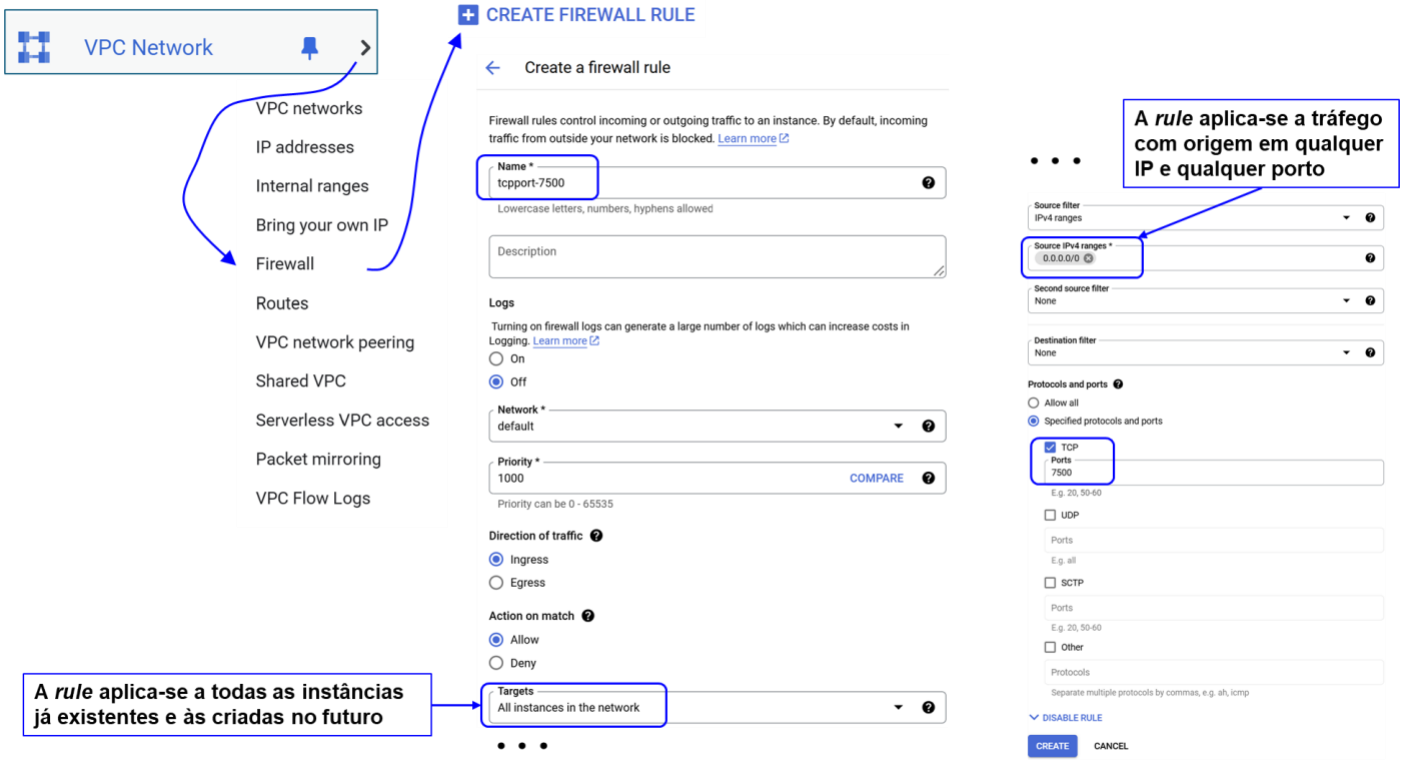


10) Usando um browser HTTP na sua máquina local, aceda ao serviço em execução na VM, experimentando as diferentes rotas com o <*host ip*> no endereço público da VM no GCP.

11) Execute outra instância do serviço na VM do GCP, usando o porto 7500:
    `java -jar ServiceREST.jar 7500`

12) Na sua máquina local execute a rota `http://<VM ip>:7500/ping` e verifique que não consegue obter resposta.

13) No seu projeto GCP crie uma *firewall rule* para permitir acessos ao porto 7500 e verifique posteriormente que já consegue aceder ao serviço a partir de qualquer computador. A seguir ilustra-se como criar uma *firewall rule* no projeto GCP.


VPC Network
VPC networks
IP addresses
Internal ranges
Bring your own IP
Firewall
Routes
VPC network peering
Shared VPC
Serverless VPC access
Packet mirroring
VPC Flow Logs
CREATE FIREWALL RULE
Create a firewall rule
Firewall rules control incoming or outgoing traffic to an instance. By default, incoming traffic from outside your network is blocked. Learn more
Name *
tcpport-7500
Lowercase letters, numbers, hyphens allowed
Description
Logs
Turning on firewall logs can generate a large number of logs which can increase costs in Logging. Learn more
On
Off
Network *
default
Priority *
1000
COMPARE
Priority can be 0 - 65535
Direction of traffic
Ingress
Egress
Action on match
Allow
Deny
Targets
All instances in the network
A rule aplica-se a todas as instâncias já existentes e às criadas no futuro
A rule aplica-se a tráfego com origem em qualquer IP e qualquer porto
Source filter
IPv4 ranges
Source IPv4 ranges *
0.0.0.0/0
Second source filter
None
Destination filter
None
Protocols and ports
Allow all
Specified protocols and ports
TCP
Ports
7500
E.g. 20, 50-60
UDP
Ports
E.g. all
SCTP
Ports
E.g. 20, 50-60
Other
Protocols
Separate multiple protocols by commas, e.g. ah, icmp
DISABLE RULE
CREATE
CANCEL

ANEXO: **Ponto 2 para computadores com Linux ou MacOS.**

ssh	Estabelecer ligação remota com terminal seguro https://linux.die.net/man/1/ssh
ssh-keygen	Geração de par de chaves pública-privada https://linux.die.net/man/1/ssh-keygen
scp	Cópia de ficheiros de e para máquina remota https://linux.die.net/man/1/scp

A. Na máquina Mac ou Linux, na diretoria *home* do utilizador corrente, gerar um novo par de chaves com o comentário **`CN2526-T1-G01`** (identificador do projeto GCP do grupo de alunos), algoritmo RSA e tamanho de chave 2048, deixando a palavra-passe vazia. O ficheiro **`~/.ssh/cn-key_rsa`** fica com a chave privada e o **`~/.ssh/cn-key_rsa.pub`** a chave pública:

```
$ ssh-keygen -f ~/.ssh/cn-key_rsa -C CN2526-T1-G01 -t rsa -b 2048
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in .../.ssh/cd-key_rsa
Your public key has been saved in .../.ssh/cd-key_rsa.pub
The key fingerprint is:
SHA256:2wiZttxX4cgl+FzHKN+CqivkRnQqMvSGLWJNWX/o8BU CD2425D-G01
The key's randomart image is:
+---[RSA 2048]----+
|                 |
|     .   E   o   |
|    o . o + = o  |
| . o...= * X +   |
|. *. oO S * = .  |
|++.=+o * = . .   |
|ooo=  o = o      |
|    +  . .       |
|   . .o.         |
+----[SHA256]-----+
```

B. Fazer *login* na máquina virtual (VM) com a chave privada criada anteriormente. No caso da VM com o IP `34.175.147.237`:

```
ssh -i ~/.ssh/cn-key_rsa CN2526-T1-G01@34.175.147.237
```

C. Para copiar um ficheiro (ex: `App.jar`) da máquina local para a VM remota com a chave privada criada anteriormente:

```
scp -i ~/.ssh/cn-key_rsa App.jar CN2526-T1-G01@34.175.147.237:~/App.jar
```